BÖLÜM 9 / MODULE 9

Eşzamanlılık Hataları, Teşhis & Düşük Seviyeli Primitifler

Concurrency Bugs, Anti-Patterns & Low-Level Primitives

Klasik yarış durumları (Check-then-Act, Read-Modify-Write), Coffman ölümcül kilitlenme (Deadlock) koşulları ve tespiti, Livelock ve Starvation, ABA problemi ve AtomicStampedReference, jstack/jcmd thread dump adli analizi, ve modern VarHandle bellek çitleri.

Classic race conditions (Check-then-Act, Read-Modify-Write), Coffman deadlock prerequisites and programmatic detection, Livelock and thread starvation, ABA memory hazard and AtomicStampedReference, production thread dump forensics with jstack/jcmd, and modern VarHandle memory fences.

49

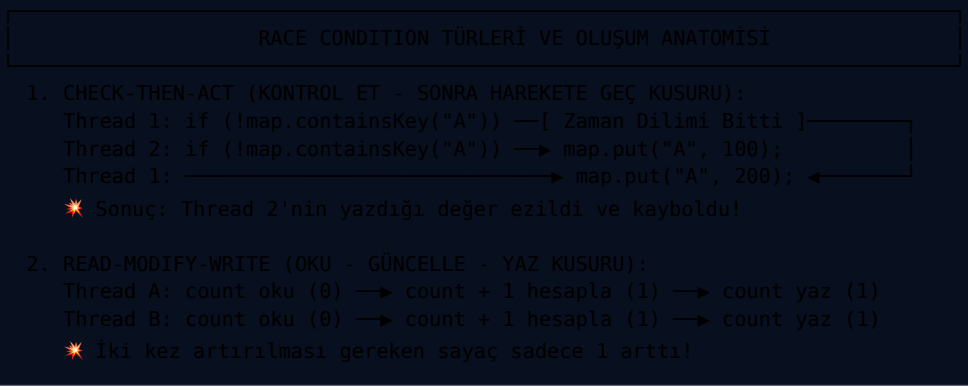
BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Klasik Yarış Durumları (Race Conditions): Check-Then-Act ve Read-Modify-Write Kusurları

Classic Race Conditions: Check-Then-Act & Read-Modify-Write Flaws

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Check-then-act, banyoya girmeden önce kapıyı tıklatıp içeride kimse olmadığını görmek, tam içeri adım atacakken başka birinin yan kapıdan hızla banyoya girmesi gibidir. Sen 'Banyo boştu!' diyerek içeri dalarsın ve felaket yaşanır! Kontrol anı ile eylem anı arasında geçen mikrosaniyelik boşlukta evren değişmiştir. Read-modify-write ise aynı anda iki kişinin buzdolabında kalan son yumurtayı görüp ikisinin de tavaya kırmasıdır; sonunda yumurta yok olur!

🇬🇧 ENGLISH (ELI5)

Check-then-Act is knocking on a dressing room door, confirming it is empty, and pausing for one second before stepping in. In that one split-second, someone else slips inside. You burst in thinking it is still vacant, causing chaos. The state observed during the 'check' was invalidated before the 'act'. Read-Modify-Write is two cooks reading an inventory sheet showing 1 egg left, both taking that egg to bake a cake simultaneously, resulting in missing inventory!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kritik Bölge Zafiyetleri ve Atomik Çözümler: 1) Check-Then-Act: En yaygın örneği `Map.containsKey()` ardından `Map.put()` yapılması veya tembel yüklemedeki (Lazy Init) `if (instance == null)` kontrolüdür. Kontrol ile yazma arasında atomik bir bağ yoktur. Eşzamanlı ortamda durum başka bir thread tarafından değiştirilebilir. Çözüm: `ConcurrentMap.putIfAbsent()` veya `computeIfAbsent()` gibi donanımsal CAS tabanlı atomik primitifler kullanmaktır. 2) Read-Modify-Write: `count++` işlemi aslında 3 bytecode komutudur (`getfield`, `iadd`, `putfield`). İki thread aynı anda `getfield` yaparsa biri diğerinin artırmasını ezer. Çözüm: `AtomicInteger.incrementAndGet()` veya `synchronized` kilididir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Critical Section Vulnerabilities: 1) Check-Then-Act: Arises when a state inspection informs a subsequent mutation, but the observed state becomes stale before the mutation executes. Common in compound collections (`if (map.containsKey(k)) map.put(k, v)`). Resolution: Leverage compound atomic methods like `ConcurrentMap.putIfAbsent()` or `computeIfAbsent()`. 2) Read-Modify-Write: Manifests in expressions like `count++`, which compile down to three discrete bytecode instructions (`getfield`, `iadd`, `putfield`). When interleaved, concurrent threads read identical initial values, discarding increments. Resolution: Atomic CAS wrappers (`AtomicInteger.incrementAndGet()`) or mutual exclusion locks.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Check-then-Act Hatası ve Modern Atomik Çözümü
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;

public class RaceConditionDemo {
    private final ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();

    // ❌ HATALI: Check-then-act yarış durumu (Compound Operation)
    public void unsafePut(String key, String value) {
        if (!cache.containsKey(key)) { // Kontrol
            // ⚠ TAM BURADA başka bir thread araya girip key'i ekleyebilir!
            cache.put(key, value);     // Eylem
        }
    }

    // ✅ DOĞRU: Tek adımda atomik işlem (Atomic Compound)
    public void safePut(String key, String value) {
        cache.putIfAbsent(key, value); // JVM ve Donanım seviyesinde atomik!
    }

    void main() {
        var demo = new RaceConditionDemo();
        try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
                final int id = i;
                pool.submit(() -> demo.safePut("CONFIG_KEY", "VAL_" + id));
            }
        }
        System.out.println("Nihai güvenli değer: " + demo.cache.get("CONFIG_KEY"));
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ConcurrentHashMap kullanmak tek başına tüm yarış durumlarını ÇÖZMEZ! Eğer haritadan okuyup (get), bir hesaplama yapıp, sonra tekrar yazıyorsanız (put), iki metodun arası atomik değildir ve yarış durumu yaşanır. Her zaman compute(), computeIfAbsent() veya merge() kullanmalısınız!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Merely wrapping state in `ConcurrentHashMap` does NOT immunize against race conditions! If your code performs `map.get()`, calculates a new value, and issues `map.put()`, the interval between calls is unguarded. You must encapsulate compound state transitions within `compute()` or `merge()`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Birden fazla adımdan oluşan işlemlerde araya başka thread girebilir. Kontrol ve eylem mutlaka tek bir atomik operasyonda birleştirilmelidir.

🎯 Key Takeaways for the Interview

• Compound actions without isolation permit thread interleaving. Merge check-and-act steps into single atomic operations.

50

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Deadlock (Ölümcül Kilitlenme) Forenziği: Coffman Koşulları, Kilit Sıralaması & ThreadMXBean

Deadlock Forensics: Coffman Conditions, Lock Ordering & JVM Detection

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki inatçı keçinin tek şeritli dar bir tahta köprüde kafa kafaya gelmesini düşünün. Biri 'Önce sen geri adım at', diğeri 'Hayır önce sen çekil' der. İkisi de yolu bırakmaz ve ikisi de köprüyü geçemez. Açlıktan ölene kadar orada dikilirler. Deadlock tam olarak budur: Thread 1, Thread 2'nin elindeki kilidi bekler; Thread 2 ise Thread 1'in elindekini bekler; sistem sonsuza dek donar!

🇬🇧 ENGLISH (ELI5)

Two stubborn goats meeting in the middle of a narrow single-lane bridge. Goat A demands Goat B back up; Goat B insists Goat A retreat first. Neither yields, neither moves forward, and both starve to death on the bridge. In Java Deadlock, Thread 1 holds Lock A and waits for Lock B; Thread 2 holds Lock B and waits for Lock A. Neither releases their lock, paralyzing both forever.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Coffman'ın 4 Ölümcül Koşulu ve Çözüm Yolları: Deadlock oluşması için 4 şartın AYNI ANDA gerçekleşmesi gerekir: 1) Mutual Exclusion (Karşılıklı Dışlama): Kaynak sadece tek bir thread tarafından tutulabilir. 2) Hold and Wait (Tut ve Bekle): Thread bir kaynağı tutarken diğerini bekler. 3) No Preemption (Zorla Alma Yok): Kaynak thread'in elinden zorla alınamaz. 4) Circular Wait (Döngüsel Bekleme): Thread'ler kapalı bir döngü zinciri oluşturur (T1->T2->T3->T1). Bu 4 şarttan HERHANGİ BİRİ kırılırsa deadlock imkânsız hale gelir! Endüstride en popüler çözüm: Kilit Sıralaması (Lock Ordering / Global Hierarchy) kuralıdır. Nesne kimliklerine (ör. Account ID veya System.identityHashCode) göre kilitler daima küçükten büyüğe sıralı alınır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Coffman Conditions & Systematic Remediation: Deadlock requires simultaneous presence of 4 prerequisites: 1) Mutual Exclusion: Resource held exclusively. 2) Hold and Wait: A thread holding a resource requests additional resources. 3) No Preemption: Resources cannot be forcibly expropriated. 4) Circular Wait: A closed chain of threads exists where each waits for a resource held by the next. Breaking ANY ONE condition prevents deadlock. Industry Standard Defense: Global Lock Ordering. By enforcing that locks are ALWAYS acquired in a strictly monotonic hierarchy (e.g. by comparing unique entity IDs or `System.identityHashCode`), circular wait becomes mathematically impossible.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Sıralı Kilitleme (Lock Ordering) ile Deadlock'tan Kaçınma
import java.util.concurrent.locks.ReentrantLock;

public class DeadlockFreeTransfer {
    record Account(long id, ReentrantLock lock) {}

    public static void transfer(Account from, Account to, double amount) {
        // Kilit sıralaması: Her zaman ID'si KÜÇÜK olanı önce kilitle!
        Account firstLock = from.id() < to.id() ? from : to;
        Account secondLock = from.id() < to.id() ? to : from;

        firstLock.lock().lock();
        try {
            secondLock.lock().lock();
            try {
                System.out.printf("Transfer yapıldı: %d -> %d tutar: %.2f%n",
                    from.id(), to.id(), amount);
            } finally {
                secondLock.lock().unlock();
            }
        } finally {
            firstLock.lock().unlock();
        }
    }

    void main() {
        var accA = new Account(1001, new ReentrantLock());
        var accB = new Account(2002, new ReentrantLock());

        // Ters yönlü transferler aynı anda gelse bile ASLA deadlock olmaz!
        transfer(accA, accB, 50.0);
        transfer(accB, accA, 30.0);
        System.out.println("Transferler güvenle tamamlandı.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

tryLock(timeout) kullanarak deadlock riskini azaltabilirsiniz; ancak kilit alınamadığında daha önce alınan kilitleri bırakmayı (unlock) unutursanız livelock yaratırsınız. Kilit sıralaması (Lock Ordering) matematiksel olarak en sağlam yoldur.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mitigating deadlocks via `tryLock(timeout)` is fragile: if a thread fails to acquire the second lock but retains the first, it may induce Livelock. Monotonic lock ordering remains the definitive architectural defense.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Deadlock döngüsel kilit beklemesinden doğar. Kilitleri daima sabit bir sıralamayla (küçük ID'den büyük ID'ye) alarak döngüyü kırın.

🎯 Key Takeaways for the Interview

- Deadlock stems from circular waiting. Enforce strict lock ordering across all execution paths to break the cycle mathematically.

51

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Livelock, Thread Starvation & Priority Inversion: Sessiz Katiller

Livelock, Thread Starvation & Priority Inversion: The Silent Killers

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                    LIVELOCK VS STARVATION VS DEADLOCK

  1. DEADLOCK (ÖLÜ):
     İki thread tamamen UYUR (BLOCKED). CPU kullanımı = %0. İlerleme = 0.

  2. LIVELOCK (AŞIRI HAREKETLİ AMA SONUÇSUZ):
     İki thread sürekli birbirine yol verir. CPU kullanımı = %100! İlerleme = 0.
     Thread 1: "Önce sen geç!" ──► Geri çekilir.
     Thread 2: "Hayır sen geç!" ──► Geri çekilir. (Sonsuz kibarlık döngüsü!)

  3. STARVATION (AÇLIK):
     Açgözlü (Greedy) yüksek öncelikli thread'ler CPU'yu bırakmaz.
     Düşük öncelikli işçi thread günlerce kuyrukta bekler, asla sıra gelmez!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Livelock, dar bir koridorda karşı karşıya gelen iki aşırı kibar insanın hikayesidir. Biri sağa kaçar, diğeri de aynı anda onun sağına kaçar. Sonra ikisi birden sola kaçar. Sürekli ter döker, adım atarlar (CPU %100 yanar) ama ikisi de koridordan geçemez! Starvation (Açlık) ise açık büfe kuyruğunda öndeki VIP müşterilerin sürekli araya tanıdıklarını alması yüzünden arkadaki garibanın akşama kadar tek bir tabak yemek alamamasıdır.

🇬🇧 ENGLISH (ELI5)

Livelock is two overly polite pedestrians trying to pass each other in a narrow corridor. Both step to the left simultaneously. Both apologize and step to the right simultaneously. They are sweating profusely and sprinting back and forth (CPU 100%), but making zero forward progress! Starvation is standing at the back of a cafeteria line where VIPs continuously cut in ahead; the poor worker at the rear starves forever without receiving a meal.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üç Sessiz Katilin Mekaniği: 1) Livelock: Genellikle deadlock'u önlemek için yazılan `tryLock` mantığının yanlış uygulanmasıyla ortaya çıkar. Thread A kilidi alamayınca kendi kilidini bırakır; Thread B de aynı anda bırakır; sonra aynı anda tekrar almaya çalışırlar. Çözüm: Çarpışma anında rastgele gecikme eklemektir (Jitter / Exponential Backoff). 2) Thread Starvation: Adil olmayan (non-fair) kilitler veya CPU'yu hiç bırakmayan sonsuz döngülü thread'ler yüzünden düşük öncelikli görevlerin kaynak bulamamasıdır. Çözüm: `ReentrantLock(true)` adil kilit modu veya adil iş kuyruklarıdır. 3) Priority Inversion (Öncelik Tersinmesi): Düşük öncelikli bir thread kilit tutarken, orta öncelikli bir thread CPU'yu kapar ve düşük önceliklinin kilidi bırakmasını engeller; böylece yüksek öncelikli thread dolaylı olarak orta öncelikli tarafından rehin alınır! Çözüm: Öncelik Devralma (Priority Inheritance) protokolleridir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Silent Killers Analyzed: 1) Livelock: Frequently induced by naive retry loops paired with `tryLock()`. Both threads detect contention, release held locks simultaneously, wait an identical fixed interval, and attempt re-acquisition, looping indefinitely at 100% CPU. Remedy: Introduce randomized truncated exponential backoff (jitter). 2) Starvation: High-priority threads or unfair synchronization mechanisms (`synchronized` or non-fair `ReentrantLock`) continuously bypass threads parked in wait queues. Remedy: Deploy fair synchronization (`new ReentrantLock(true)`) or work-stealing schedulers. 3) Priority Inversion: Occurs when a low-priority thread holds a shared lock, an un-synchronized medium-priority thread preempts it on the CPU, preventing the low-priority thread from releasing the lock needed by a stalled high-priority thread. Remedy: Priority Inheritance Protocols.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Rastgele Gecikme (Jitter) ile Livelock'u Engelleme
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.locks.ReentrantLock;

public class LivelockResolutionDemo {
    private final ReentrantLock lockA = new ReentrantLock();
    private final ReentrantLock lockB = new ReentrantLock();

    public boolean executeWithJitter(String workerName) throws InterruptedException {
        while (true) {
            if (lockA.tryLock()) {
                try {
                    if (lockB.tryLock()) {
                        try {
                            System.out.println(workerName + " iki kilidi de aldı, kritik iş
yapılıyor!");
                            return true;
                        } finally {
                            lockB.unlock();
                        }
                    }
                } finally {
                    lockA.unlock();
                }
            }

            // ⚠ LIVELOCK ÇÖZÜMÜ: Eşzamanlı çakışmada rastgele bekle (Jitter Backoff)
            int jitterMs = ThreadLocalRandom.current().nextInt(10, 50);
            System.out.printf("%s kilit alamadı, %d ms rastgele uyuyor...%n", workerName, jitterMs);
            Thread.sleep(jitterMs);
        }
    }

    void main() throws Exception {
        Thread.ofVirtual().start(() -> {
            try { executeWithJitter("Worker-1"); } catch (Exception _) {}
        });
        Thread.ofVirtual().start(() -> {
            try { executeWithJitter("Worker-2"); } catch (Exception _) {}
        });
        Thread.sleep(200);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ReentrantLock'ı her zaman adil modda (fair=true) başlatmak performansı dramatik şekilde düşürebilir (throughput 10x-20x azalabilir)! Açlık sorunu gerçekten kanıtlanmadığı sürece varsayılan haksız (non-fair) modu koruyun.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Enabling fair mode (`new ReentrantLock(true)`) imposes severe throughput degradation (often 10x-20x slower) due to context switches and mandatory FIFO queuing. Reserve fair locks strictly for workloads where starvation is an empirically proven threat.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Livelock %100 CPU harcayıp ilerleyememektir; rastgele jitter ile çözülür. Açlık (starvation) ise bazı thread'lere hiç sıra gelmemesidir.

🎯 Key Takeaways for the Interview

• Livelock spins at 100% CPU without progress; resolve via randomized jitter. Starvation occurs when unfair queues starve low-priority threads.

52

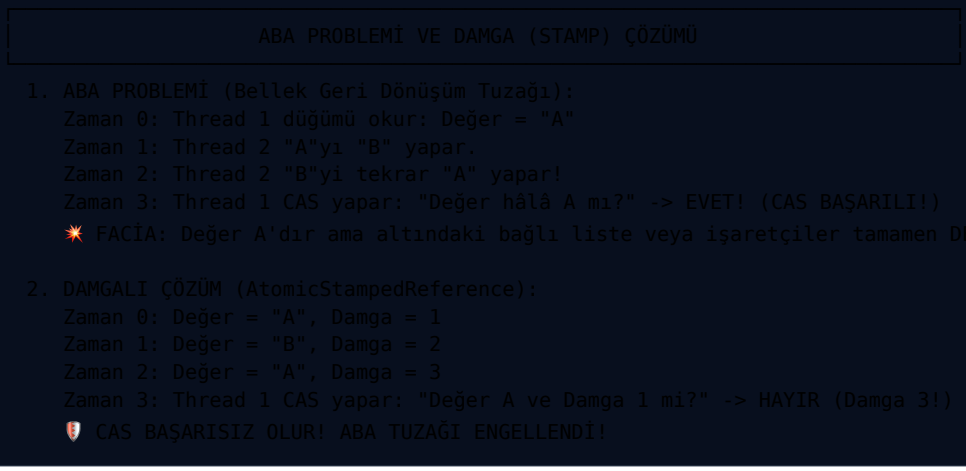
BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

ABA Problemi & Damgalı Referanslar: AtomicStampedReference vs AtomicMarkableReference

The ABA Problem & Stamped References: AtomicStampedReference vs Markable

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Masanın üstünde 100 liralık bir banknot bıraktınız. Siz kahve almaya gidince arkadaşınız o parayı alıp kumarda harcadı, sonra cebinden çıkarıp masaya yepyeni başka bir 100 liralık banknot koydu. Döndüğünüzde 'Aaa param burada duruyor' dersiniz ama o para aslında o para değildir! Arada koskoca bir kumar dönmüştür. AtomicStampedReference paranın üzerine görünmez bir versiyon damgası basar; para aynı kalsa bile damga değiştiği için kandırılmazsınız!

🇬🇧 ENGLISH (ELI5)

You leave a $100 bill on your desk. While you grab coffee, a thief steals it, gambles it away, wins, and places a completely different $100 bill back on your desk. You return and see '$100', falsely assuming nothing happened. In concurrent memory, the pointer value returned to 'A', but internal memory references were altered! `AtomicStampedReference` pairs the reference with an incrementing integer version stamp. Even if the value reverts to 'A', the mismatched stamp (version 3 vs version 1) halts the CAS instantly.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ABA Problemi ve Kilit-Siz (Lock-Free) Veri Yapıları: 1) Ortaya Çıkış: Lock-free yığınlarda (Treiber Stack) bir düğüm çıkarılırken (pop) üstteki düğüm A, onun altındaki B'dir. Thread 1, A'yı okur ve B'yi yeni baş yapmayı planlar. Araya giren Thread 2, A'yı ve B'yi çıkarır, sonra hafızadan geri dönüştürülen A nesnesini C'nin üstüne tekrar koyar. Thread 1 uyandığında yığının tepesinde hâlâ A'yı görür ve CAS yapar; yığın kafası B'ye geçer ancak B çoktan silinmiştir! Bellek bozulur. 2) AtomicStampedReference<V>: Referans ile int tipinde bir damgayı (`stamp`) atomik bir çift olarak tutar. `compareAndSet(expectedRef, newRef, expectedStamp, newStamp)` ile hem referansın hem damganın uyuştuğunu doğrular. 3) AtomicMarkableReference<V>: Damga yerine tek bitlik bir boolean (`mark`) tutar; nesnenin silinmek üzere işaretlendiğini (logical deletion) belirtmek için idealdir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ABA Hazard in Lock-Free Data Structures: 1) The Mechanics: In a lock-free Treiber Stack, Thread 1 observes top node `A` pointing to `B`. Before CAS completes, Thread 2 pops `A` and `B`, and then pushes a recycled instance `A` pointing to `C`. Thread 1 resumes and executes `CAS(top, A, B)`. The CAS succeeds because `top == A`, yet node `B` points to unallocated or corrupted memory. 2) `AtomicStampedReference\<V>`: Encapsulates both an object reference and an integer version counter. It invokes a dual-variable CAS (`compareAndSet(expectedRef, newRef, expectedStamp, newStamp)`). 3) `AtomicMarkableReference\<V>`: Pairs a reference with a single boolean mark, optimal for marking nodes as logically deleted before physical linked-list unlinking.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: AtomicStampedReference ile ABA Güvenli Hesap Güncellemesi
import java.util.concurrent.atomic.AtomicStampedReference;

public class AbaPreventionDemo {
    record AccountState(String status) {}

    void main() {
        var initial = new AccountState("ACTIVE");
        int initialStamp = 1;

        // Referans ve başlangıç versiyon damgası
        var accountRef = new AtomicStampedReference<>(initial, initialStamp);

        // Thread 1: O anki referansı ve damgayı okur
        int[] stampHolder = new int[1];
        AccountState observed = accountRef.get(stampHolder);
        int observedStamp = stampHolder[0];

        // Araya giren Thread 2: A -> B -> A döngüsü yapar (Damga artar!)
        var suspended = new AccountState("SUSPENDED");
        accountRef.compareAndSet(observed, suspended, observedStamp, observedStamp + 1);
        accountRef.compareAndSet(suspended, observed, observedStamp + 1, observedStamp + 2);

        System.out.println("Şu anki referans: " + accountRef.getReference().status());
        System.out.println("Şu anki güncel damga: " + accountRef.getStamp());

        // Thread 1 geri döner ve eski damgayla güncellemeye çalışır
        var closed = new AccountState("CLOSED");
        boolean success = accountRef.compareAndSet(observed, closed, observedStamp, observedStamp +
1);

        System.out.println("Thread 1 CAS başarılı oldu mu? " + success + " (ABA Engellendi!)");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

AtomicStampedReference dahili olarak her güncellemede yeni bir küçük nesne (Pair<V, Integer>) oluşturur! Çok yüksek yazma hacimli sistemlerde bu durum GC baskısı yaratabilir. Yüksek performanslı kütüphaneler (ör. Disruptor) damgayı long içinde bit maskeleme ile saklar.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`AtomicStampedReference` instantiates an internal `Pair\<V, Integer>` node upon state mutations, inducing GC allocation overhead under extreme write contention. Low-latency engines mitigate this by packing references and monotonic counters into a single 64-bit primitive `long` via bit shifting.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CAS sadece değerin eşitliğini kontrol eder, arada değişip geri gelmesini anlayamaz. Versiyon damgası (stamp) ekleyerek ABA hatasını önleyin.

🎯 Key Takeaways for the Interview

• CAS inspects current value equality, blind to intermediary transitions. Coupling references with monotonic version stamps prevents the ABA trap.

53

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Production Thread Dump Analizi: jstack, jcmd, Thread Durumları & CPU Tavanlarını Teşhis

Production Thread Dump Analysis: jstack, jcmd & Diagnosing CPU Spikes

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                PRODUCTION THREAD DUMP ADLİ TIP REHBERİ       │
└──────────────────────────────────────────────────────────────┘

  CANLI SUNUCUDA CPU %100 VEYA KİLİTLENME VARSA:

  1. PID TESPİTİ:    top -H -p <PID> ──► En çok CPU yiyen Thread ID'yi bul (ör. 12948)
  2. HEX DÖNÜŞÜMÜ:   printf '%x\n' 12948 ──► "0x3294" (nid formatı)
  3. DUMP ALMA:      jcmd <PID> Thread.print > dump.tdump (veya jstack -l <PID>)
  4. NID ARAMA:      grep -A 20 "0x3294" dump.tdump ──► Suçlu metod ve satır karşında!

  DUMP'TA NE ARANIR?
  - "waiting to lock <0x00...>" ──► Kilidi kim tutuyor?
  - "locked <0x00...>"          ──► Kilidi tutan thread ne yapıyor?
  - "Found 1 deadlock."         ──► HotSpot deadlock'u otomatik listeler!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Thread dump, bir binadaki yangın anında bütün çalışanların dondurulup röntgenlerinin çekilmesidir. Kim elinde yangın tüpüyle bekliyor, kim kapıyı arkadan kilitlemiş, kim koridorda boşuna koşup terliyor tek bir fotoğrafta görünür. `top` komutuyla en çok terleyen personeli bulursun, thread dump ile de o personelin tam olarak hangi odada hangi kâğıdı imzalamaya çalıştığını saniyesi saniyesine yakalarsın!

🇬🇧 ENGLISH (ELI5)

A Thread Dump is an instantaneous radiographic snapshot of every single worker in your distributed factory. It captures who is sleeping, who is fighting over a wrench, and who is frantically running on a treadmill. Using `top -H` you pinpoint the exact OS thread ID consuming 100% CPU, convert it to hexadecimal (`nid`), and search the thread dump to reveal the exact Java file and line number executing the rogue loop.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Thread Dump Anatomisi ve Teşhis Teknikleri: 1) Üretim Komutları: Modern JDK'larda `jstack` yerine `jcmd \<PID> Thread.print -l` tercih edilir. Sanal thread'leri dökmek için `jcmd \<PID> Thread.dump_to_file -format=json dump.json` kullanılır. 2) Dump Başlığı Çözümleme: `"http-nio-8080-exec-12" #45 daemon prio=5 os_prio=0 tid=0x7f nid=0x3294 waiting on condition [0x7000]`. Burada `nid` (Native Thread ID) işletim sistemi thread ID'sinin hex karşılığıdır. 3) Kilit Zincirleri: `java.lang.Thread.State: BLOCKED (on object monitor)` görüyorsanız `waiting to lock \<0x00007f...>` adresini dump içinde aratıp o adresi `locked \<0x00007f...>` olarak elinde tutan thread bulunur. 4) Deadlock Tespiti: HotSpot JVM dökümün en altında `Found one Java-level deadlock` başlığıyla kilit döngüsünü otomatik raporlar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Thread Dump Anatomy & Diagnostic Protocol: 1) Modern Extraction Tooling: Prefer `jcmd \<PID> Thread.print -l` over legacy `jstack`. To analyze virtual threads, invoke `jcmd \<PID> Thread.dump_to_file -format=json threads.json`. 2) Header Dissection: In `"pool-1-thread-1" #12 prio=5 os_prio=0 cpu=142.12ms nid=0x1a4b waiting on condition`, `nid` is the native OS thread ID in hexadecimal. By mapping CPU-heavy PIDs from `top -H -p \<PID>` to hexadecimal, you identify the exact hotspot method trace. 3) Monitor Chain Resolution: Search for `waiting to lock \<0xADDR>` and grep for `locked \<0xADDR>` to construct lock dependency graphs. 4) JVM Deadlock Engine: HotSpot automatically scans for cyclic object monitor dependencies and appends a formal deadlock report to the dump tail.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Programatik Olarak Deadlock Tespiti ve Thread Dökümü Alma
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;

public class ThreadDumpDiagnosticDemo {
    public static void scanForDeadlocks() {
        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] deadlockedThreadIds = bean.findDeadlockedThreads();

        if (deadlockedThreadIds != null && deadlockedThreadIds.length > 0) {
            System.err.println("🚨 KRİTİK ALARM: " + deadlockedThreadIds.length + " Thread Deadlock'a
girdi!");
            ThreadInfo[] infos = bean.getThreadInfo(deadlockedThreadIds, true, true);
            for (ThreadInfo info : infos) {
                System.err.printf("Deadlock Thread: %s (ID: %d) beklediği kilit: %s, kilidi tutan:
%s%n",
                    info.getThreadName(), info.getThreadId(),
                    info.getLockInfo(), info.getLockOwnerName());
            }
        } else {
            System.out.println("✅ Deadlock tespit edilmedi. Sistem sağlıklı.");
        }
    }

    void main() {
        scanForDeadlocks();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Production ortamında asla tek bir thread dump ile karar vermeyin! 10 saniye arayla en az 3 adet thread dump alın (ör. dump1, dump2, dump3). Eğer bir thread 3 dökümde de aynı satırda RUNNABLE durumundaysa sonsuz döngüdedir; eğer 3 dökümde de BLOCKED ise kilit açlığını gösterir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Never diagnose a production outage from a single thread dump! Always capture at least 3 consecutive snapshots spaced 5-10 seconds apart. A thread observed as `RUNNABLE` at the identical instruction line across all snapshots is trapped in an infinite loop; persistent `BLOCKED` states confirm lock starvation.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• top -H ile yüksek CPU kullanan thread ID'yi hex'e çevirin, jcmd Thread.print çıktısında 'nid' olarak aratıp sorunu nokta atışı tespit edin.

🎯 Key Takeaways for the Interview

• Convert high-CPU OS thread IDs to hex via `printf '%x'`, grep for `nid` in `jcmd Thread.print`, and trace the exact offending call stack.

54

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

VarHandle & Düşük Seviyeli Bellek Çitleri: Modern Java'da Unsafe'in Yerine Geçiş

Low-Level VarHandle & Memory Fences: Replacing sun.misc.Unsafe

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                VARHANDLE BELLEK ERİŞİM MODLARI               │
└──────────────────────────────────────────────────────────────┘

Mod Adı            | Bellek Semantiği                | CPU Bariyeri
-------------------|---------------------------------|-----------------------
get() / set()      | Plain (Düz Okuma/Yazma)         | SIFIR Bariyer (En Hızlı)
getOpaque()        | Opaque (Derleyici optimiz. yok) | Reordering yok, bariyer yok
getAcquire()       | Acquire (Okuma Bariyeri)        | LoadLoad + LoadStore
setRelease()       | Release (Yazma Bariyeri)        | StoreStore + LoadStore
getVolatile()      | Volatile (Tam Sıralama)         | StoreLoad (En Katı)

Atomic RMW:
handle.compareAndSet(this, expected, update) ──► Donanımsal CMPXCHG
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski `Unsafe` sınıfı, eline dinamit verilmiş bir inşaat işçisi gibidir. Çok hızlıdır ama yanlışlıkla duvar yerine ana taşıyıcı kolonu patlatıp tüm JVM'i çökertebilir. Java 9 ve 25'teki `VarHandle` ise koruyucu kaskı, lazer hizalayıcısı ve sigortası olan yüksek teknolojili bir cerrahi alettir! Belleğe doğrudan, kilit almadan, ışık hızında dokunmanı sağlar ama hafızayı bozamayacağın güvenlik bariyerleriyle donatılmıştır.

🇬🇧 ENGLISH (ELI5)

Legacy `sun.misc.Unsafe` was handing a construction worker live dynamite. It bypassed all safety checks, but a 1-byte off-by-one pointer error crashed the JVM instantly with a Segfault. `VarHandle` (Java 9 - Java 25) is a precision laser scalpel. It offers the exact same raw, zero-allocation hardware access and atomic memory fences as `Unsafe`, but is fully type-safe, bounds-checked, and natively optimized by the JIT compiler.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

VarHandle Bellek Modeli ve Erişim Semantikleri: 1) Unsafe'in Emekliliği: `sun.misc.Unsafe` doğrudan bellek adresleriyle çalıştığı için JVM güvenliğini (Memory Safety) ihlal ediyordu. `java.lang.invoke.VarHandle`, MethodHandles API'si üzerinden değişkenlere atomik ve düşük seviyeli erişim sunar. 2) 5 Farklı Erişim Modu: a) Plain: Standart alan erişimi, reordering serbesttir. b) Opaque: Bytecode döngülerinde değişkenin önbelleğe alınmasını (hoisting) engeller, atomiktir ancak bellek bariyeri koymaz. c) Acquire/Release: Tek yönlü bariyerlerdir. `setRelease` öncesindeki yazmaların bu yazmadan sonra görünmesini garanti eder; tam `volatile` kadar pahalı değildir. d) Volatile: Çift yönlü tam bellek bariyeri (StoreLoad). 3) Düşük Seviyeli Bellek Çitleri: `VarHandle.fullFence()`, `acquireFence()`, `releaseFence()` gibi metotlarla CPU düzeyinde donanımsal bellek bariyerleri manuel olarak da tetiklenebilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

VarHandle Architecture & JVM Memory Modes: 1) Superseding `sun.misc.Unsafe`: `Unsafe` permitted raw memory dereferencing, provoking hard JVM segmentation faults. `VarHandle` integrates with `MethodHandles.Lookup`, enforcing variable type safety and reference validation while compiling down to native assembly intrinsics. 2) Granular Memory Access Modes: a) Plain: Normal reads/writes without fences. b) Opaque: Bitwise atomic with program-order coherence, preventing compiler hoisting while avoiding bus locks. c) Acquire/Release: One-way memory barriers. `setRelease` flushes preceding writes; `getAcquire` guarantees subsequent reads see updated state without incurring total StoreLoad fence penalties. d) Volatile: Enforces sequential consistency with global `StoreLoad` barriers. 3) Native CPU Fences: Exposes hardware primitives via `VarHandle.fullFence()` and `loadLoadFence()` for ultra-low-latency frameworks (e.g. Aeron, Disruptor).

⚡ Modern Java 25 LTS Kodu / ⚡ Modern Java 25 Implementation

```
// Java 25: VarHandle ile Kilit-Siz Atomik Sayaç ve Acquire/Release Semantiği
import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

public class VarHandleCounter {
    private volatile int count = 0;

    // VarHandle tanımlayıcısı
    private static final VarHandle COUNT_HANDLE;

    static {
        try {
            // Field erişim kontrolü ve VarHandle oluşturma
            MethodHandles.Lookup lookup = MethodHandles.lookup();
            COUNT_HANDLE = lookup.findVarHandle(VarHandleCounter.class, "count", int.class);
        } catch (ReflectiveOperationException e) {
            throw new ExceptionInInitializerError(e);
        }
    }

    public void incrementRelaxed() {
        // Opaque veya Release modunda artırma (Volatile'dan daha ucuz CPU döngüsü!)
        int current;
        do {
            current = (int) COUNT_HANDLE.getOpaque(this);
        } while (!COUNT_HANDLE.compareAndSet(this, current, current + 1));
    }

    public int getAcquire() {
        return (int) COUNT_HANDLE.getAcquire(this);
    }

    void main() {
        var counter = new VarHandleCounter();
        counter.incrementRelaxed();
        System.out.println("VarHandle ile güncellenen sayaç: " + counter.getAcquire());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı / ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

VarHandle tanımlayıcıları (MethodHandles.Lookup) MUTLAKA `private static final` olarak tanımlanmalıdır! Eğer nesne düzeyinde (instance variable) veya metot içinde lookup yaparsanız JIT derleyicisi çağrıyı inline edemez ve performans 100 kat düşer.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`VarHandle` references MUST be declared `private static final`! If lookups are performed dynamically inside instance methods, the JIT C2 compiler cannot constant-fold the callsite into direct native assembly intrinsics, causing a catastrophic 100x performance drop.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar / 🎯 Key Takeaways for the Interview

- VarHandle, Unsafe'in güvenli ve modern halidir. Plain, Opaque, Acquire/Release ve Volatile modlarıyla CPU bariyerlerini milimetrik kontrol etmenizi sağlar.

- VarHandle replaces `sun.misc.Unsafe` with type-safe, JIT-intrinsic memory operations across Plain, Opaque, Acquire/Release, and Volatile barrier modes.